KnowVault 运维与故障排查手册

1 摄入失败与重试

文档摄入走 MassTransit 的 document-ingest 端点，该端点显式配置了消息重试策略，参数是五次重试、每次间隔两秒。重试次数耗尽后消息进入死信队列 document-ingest_error，此时文档状态被落成 Failed，错误原因写入文档记录的 ErrorMessage 字段。

消费者在独立的业务工作单元里落失败状态后再把异常抛出，这样即使整个消费被判定为失败，前端仍然能看到失败结论而不是永远停在处理中。

排查顺序建议：先确认 RabbitMQ 里死信队列是否堆积，再看宿主日志中的解析异常，最后用重新处理接口重试单个文档。如果只是想清空向量重跑，直接调用重新处理即可，它会先清理旧向量与旧切片。

2 依赖服务与端口

本机验收路径不依赖 Aspire，可以直接复用手工容器 kv-postgres、kv-redis、kv-qdrant、kv-rabbitmq 与 kv-minio，用 docker start 加通配符一次拉起，数据保存在容器可写层。仓库里没有 docker-compose 文件，所有配置通过环境变量覆盖。

端口对应关系需要特别注意三处。PostgreSQL 在本机映射到宿主 5433，与代码默认连接串里的 5432 不一致。Qdrant 客户端走 gRPC 端口 6334，不是 REST 的 6333，用错端口时表现为连接成功但调用超时。Seq 的 HTTP 地址是 5341。

文档状态是一个四值状态机，取值依次为 Pending、Processing、Ready 与 Failed。上传成功后是 Pending，消费者取到消息后置为 Processing，解析与向量化全部完成置为 Ready，任何一步抛异常则落 Failed 并把原因写进错误消息字段。重新处理会把状态重置回 Pending，因此并非只能单向推进。前端不订阅进度推送，而是每两秒轮询一次文档列表来刷新状态，所以状态变化到界面上最多有约两秒延迟，这不等于摄入卡住。

向量集合的初始化由一个宿主启动服务负责，启动时幂等地确保集合存在。所以反复重启不会重复建集合，也不会清空已有向量。这个初始化失败时只记录错误日志、不阻断应用启动，设计意图是让检索与写入在各自调用时再暴露问题，因此启动日志里出现一次初始化失败并不代表服务不可用。如果确实要重建索引，正确做法是调用文档的重新处理接口逐个清理，而不是手工删集合。

3 可观测性

日志链路是 Serilog 写文件与控制台，同时经 Seq sink 直送结构化日志。链路追踪用 OpenTelemetry 的 OTLP HTTP protobuf 协议发送到 Seq 的 traces 摄入路径，原因是 Seq 在本地跑 gRPC 需要启用 HTTPS，明文场景只能走 HTTP。

EF Core 的插桩包版本被锁在 1.17.0-beta.1，这是一个预发布版本但目标框架是 net10.0，属有意选择，不要随意升级或降级。自定义 ActivitySource 的名称是 KnowVault，一次问答会依次产生 qa.ask、rag.retrieve、embedding.embed 与 llm.complete 四个 span，开启重排后多出 rerank.rerank，文档摄入产生 ingest.process。

4 认证相关的表现差异

未授权访问在 API 前缀下返回 401 并带 Bearer 质询头，浏览器直接访问其它路径则跳转登录页。看到接口返回 302 而不是 401，通常说明请求路径没有落在 API 前缀下，或者改写中间件注册顺序不对。

SignalR 集线器的令牌只能放查询字符串，服务端由一层专用中间件把它搬运成 Bearer 头，该中间件必须注册在认证管道之前。集线器握手失败先查这一步。

5 权限变更后不生效

角色与知识库成员的判定结果存在缓存。修改成员或角色之后，需要重新登录获取新令牌，或者等待缓存过期，否则接口仍然按旧权限返回 403。这是验收权限用例时最常见的假故障。

日志的最小级别在配置文件里按命名空间分别设定：默认级别是 Information，Microsoft 命名空间保持 Information，而 EntityFrameworkCore 被降到 Warning，原因是 EF 生成的 SQL 日志量太大会淹没有效信息。文件日志的落盘路径是应用目录下的 Logs 文件夹里的 logs 文本文件，所有写入目标都包在异步 sink 里，避免磁盘慢时阻塞请求线程。

追踪的父子结构不需要手工串联。问答入口先开启 qa.ask 这个最外层 span，其后在同一调用链里由检索、嵌入、重排与大模型服务各自开启自己的 span，运行时按当前活动上下文自动把它们挂成父子关系。文档摄入则自成一个 ingest.process 链，与问答不属于同一条追踪。

6 检索结果异常

检索为空而文档状态是就绪时，按三个方向排查。第一是租户标识归一化：单租户运行时空租户会被写成固定零值Guid，若某个写入路径漏了归一化，Qdrant 的租户过滤条件永远不命中。第二是集合与维度：向量集合名是 knowvault_chunks，维度必须与嵌入模型输出一致，为 1024，维度不匹配时写入直接失败。第三是知识库过滤：检索按租户加知识库两个条件过滤，问答时选错知识库会表现为查不到。

重排异常的表现是答案质量下降而不是报错。重排接口失败或返回空时按放行策略退回向量顺序，因此日志里没有异常并不代表重排成功，需要看 Seq 中 rerank.rerank 这个 span 的状态与标签。

重排必须使用独立的基础地址配置，它是 DashScope 的专有接口，不属于 OpenAI 兼容模式。如果复用了嵌入模型那个兼容模式地址，会收到明确的接口不存在错误。旧的重排模型名已下线，当前默认模型是 qwen3-rerank。

7 测试环境说明

集成测试用 Testcontainers 拉起 PostgreSQL、RabbitMQ、MinIO 与 Qdrant 四个容器，嵌入模型与消息发布用替身实现。服务级测试继承 ABP 的集成测试基类，用始终放行的授权配置绕过权限拦截器，但知识库成员表的真实校验仍然保留。

本机如果拉取 MinIO 镜像受限，可以设置环境变量把对象存储切换为内存实现兜底；持续集成环境不允许设置该变量，必须跑真实 MinIO 容器。

8 持续集成与代码规范检查

持续集成流水线由三个作业组成。后端作业负责构建、单元测试与集成测试，并且把构建警告当作错误处理；前端作业依次执行依赖安装与打包构建；格式作业跑分析器风格的格式检查，并显式只关心枚举命名这一条诊断编号。

这里有一个容易踩空的点：枚举类型名必须以 Enum 结尾这条规则属于命名类诊断，而命名规则不参与编译构建，即使打开了在构建时检查代码风格的开关也不会拦下违规，必须单独跑格式检查命令才能检出。本地构建通过不等于格式检查通过。

另一条只能靠人工保证的规则是行尾统一。仓库要求文本文件统一使用 Windows 行尾，同一个文件混用两种行尾会触发编辑器的标准化弹窗。但 git 的属性配置会在写库时将行尾归一化，因此行尾混用在状态列表和差异视图里完全看不出来，必须逐字节检查文件内容才能确认。

9 上传相关的应用层限制

上传的扩展名白名单与单文件大小上限属于应用层配置，默认允许七种扩展名并且上限是 50 兆字节。不在白名单或超限都会抛出面向用户的友好异常，返回的是中文提示而不是堆栈，因此这类失败在 Seq 里看不到错误级别日志，属于预期行为。